

Unit 1

Complete Study Notes & Exam Revision Guide

Overview of OS • Objectives & Functions • Kernel & Shell • Multiprogramming & Multitasking • CPU Scheduling • FCFS • Round Robin • SVR3 (Traditional UNIX) Scheduling

Unit	Teaching Hours	Exam Weight	Prepared For
Unit 1 — OS Fundamentals	9 Hrs (as per syllabus)	Mid-Semester — 10 Marks	AMCA Semester Exam

What's inside this guide

11 structured sections | Solved Gantt-chart numericals for FCFS & Round Robin | Worked example of the SVR3 priority formula | Comparison tables | A formula sheet | Exam-style Q&A revision | Glossary of key terms

Prepared for exam revision. Read alongside your class notes and textbook (Silberschatz & Galvin, Operating System Concepts; Stallings, Operating Systems) for the fullest depth of coverage.

Contents & Study Roadmap

- 1 Introduction to Operating Systems — definition, two views, objectives, functions
- 2 Kernel & Shell — types of kernel, types of shell, kernel vs shell, system calls
- 3 Multiprogramming Operating Systems — concept, working, pros & cons
- 4 Multitasking Operating Systems — preemptive vs cooperative, vs multiprogramming
- 5 CPU Scheduling — Concepts — schedulers, criteria, preemptive vs non-preemptive
- 6 FCFS Scheduling — algorithm, solved example, Gantt chart, pros & cons
- 7 Round Robin Scheduling — time quantum, solved example, Gantt chart, pros & cons
- 8 SVR3 Scheduling Algorithm — priority bands, formula, worked numeric trace
- 9 Quick Comparison — FCFS vs Round Robin vs SVR3
- 10 Exam-Focused Revision — Q&A and the Formula Sheet
- 11 Glossary of Key Terms & Study Tips

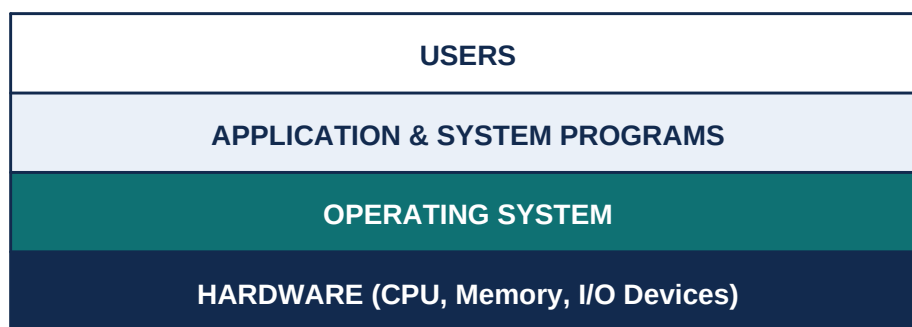
How to use this guide: Sections 1–5 build the conceptual foundation (definitions, functions, classifications). Sections 6–8 are numerical/algorithmic — these are where 10-mark questions with Gantt charts usually come from, so work through the solved examples with a pen and paper rather than just reading them. Section 10 is a fast pre-exam revision pass.

1.1 What is an Operating System?

An **Operating System (OS)** is system software that acts as an intermediary between the computer user / application programs and the computer hardware. It manages all hardware and software resources of the computer and provides common services that application programs need in order to run.

An OS can be understood from two complementary points of view:

- **User view (Extended / Virtual Machine):** the OS hides the messy, low-level details of the hardware (interrupts, timers, raw memory addresses, device registers) and presents the user with a clean, convenient interface — files, folders, windows, programs — that is easier to use than the bare machine.
- **System view (Resource Manager):** the OS is the manager of all hardware resources — CPU, main memory, I/O devices, secondary storage — and decides how these limited resources are allocated fairly and efficiently among the many programs and users competing for them.



Layered view: the OS sits between the raw hardware and the programs/users, exposing hardware capability upward while hiding hardware complexity.

Common examples: Windows, Linux/UNIX, macOS, Android, iOS.

1.2 Objectives of an Operating System

Every general-purpose OS is designed around three core objectives:

1. Convenience — make the computer system convenient and easier to use. The OS hides hardware complexity behind simple abstractions (files, processes, windows) so users and programmers don't need to manage hardware directly.

2. Efficiency — manage the hardware resources of the computer system efficiently. This includes maximizing CPU utilization, using memory effectively, and ensuring fair, optimal use of I/O devices among competing processes.

3. Ability to Evolve — permit the effective development, testing and introduction of new system functions without interfering with existing service. This covers extensibility, portability across hardware, and backward compatibility as the system grows.

*Note: some textbooks list a fourth objective — **Throughput** — maximizing the amount of useful work the system completes per unit time; this overlaps closely with efficiency.*

1.3 Functions of an Operating System

Acting as the system's resource manager, the OS performs the following key functions:

Function	What it does
Process Management	Creation, scheduling, suspension, resumption, termination and synchronization of processes.
Memory Management	Allocating and deallocating main memory space to processes; keeping track of which memory is free/used.
File Management	Creation, deletion, reading, writing, and organizing files and directories; controlling access to them.
I/O / Device Management	Managing input/output devices through device drivers; buffering, caching and spooling of I/O data.
Secondary-Storage Management	Disk space allocation, disk scheduling, and free-space management on secondary storage.
Security & Protection	Authenticating users and controlling access to resources so one process/user cannot harm another.
Networking / Communication	Managing data exchange between processes on the same or different networked machines.
Command Interpretation	Providing a shell/CLI or GUI so users can issue commands to the system.
Error Detection & Handling	Continuously monitoring the system for errors (hardware/software) and taking corrective action.
Resource Allocation & Accounting	Allocating resources fairly among multiple users/jobs and keeping usage records/logs.

2.1 The Kernel

Kernel — the core component of the operating system. It is loaded into memory at boot time, stays resident for as long as the system runs, and has complete, privileged control over the hardware. It directly manages the CPU, memory, and devices, and handles all system calls and interrupts.

The kernel runs in a privileged **kernel mode** (unrestricted access to hardware), while ordinary application programs run in a restricted **user mode**. A process must switch into kernel mode (via a system call) whenever it needs a privileged service such as file I/O or memory allocation.

Types of Kernel

Type	Structure	Advantage	Disadvantage	Example
Monolithic Kernel	Entire OS (process mgmt, memory mgmt, file system, device drivers) runs as one large program in kernel space.	Fast — no message-passing overhead.	A bug in any component can crash the whole system.	Traditional UNIX, Linux
Microkernel	Only the bare essentials (IPC, basic scheduling, minimal memory mgmt) run in kernel space; file systems and drivers run as user-space servers.	More stable and secure; a crashing service doesn't crash the kernel.	Slower, due to frequent message passing between user-space servers.	Minix, QNX
Hybrid Kernel	Combines monolithic and microkernel ideas — some services kept in kernel space for speed, others kept modular.	Balance of performance and structure.	More complex design.	Windows NT family, macOS (XNU)

2.2 The Shell

Shell — the outermost layer of the OS; a command interpreter that provides the interface between the user and the kernel. It accepts commands typed (or clicked) by the user and translates them into system calls / instructions that the kernel can execute.

- **Command Line Interface (CLI) shells** — accept typed text commands. Examples: **sh**, **bash**, **csh**, **ksh** (UNIX/Linux); **cmd.exe**, PowerShell (Windows).
- **Graphical User Interface (GUI) shells** — accept commands through windows, icons, menus and pointers. Examples: Windows Explorer, GNOME, KDE.

2.3 Kernel vs Shell

Aspect	Kernel	Shell
Definition	Core of the OS; manages hardware resources directly.	Interface layer; interprets user commands.
Location	Innermost layer, closest to hardware.	Outermost layer, closest to the user.
Mode	Runs in privileged kernel mode.	Runs in user mode.
Function	Resource management (process, memory, device, file).	Command interpretation and dispatch.
Interacts with	Hardware directly, and the shell/programs indirectly via system calls.	The user (input/output) and the kernel (via system calls).
Replaceable?	Not normally replaced by the user.	User can usually choose/change shells (e.g. bash → zsh).

2.4 How They Interact — System Calls

When a user types a command in the shell, the shell parses it and invokes the appropriate **system call(s)** on behalf of the user. The kernel executes the privileged request and returns the result to the shell, which then displays the output to the user. System calls are therefore the well-defined programming interface through which user-mode programs (including the shell) request kernel-mode services.

3.1 Concept

Multiprogramming — the ability of an OS to keep more than one program (job) in main memory at the same time, so that when the currently running job needs to wait for a slow I/O operation, the CPU is switched to another ready job instead of sitting idle.

Early **uniprogramming** systems ran one job at a time and left the CPU idle for long stretches whenever that job waited on I/O — a major waste of an expensive resource. Multiprogramming was introduced specifically to overlap the I/O of one job with the CPU execution of another, raising overall CPU utilization.

3.2 How Multiprogramming Works

1. Several jobs are kept ready in a **job pool** in main memory.
2. The OS picks one job and gives it the CPU.
3. When that job issues an I/O request and must wait, the OS performs a **context switch** and hands the CPU to another ready job.
4. This continues so the CPU is rarely left idle, as long as at least one job is ready to run.

3.3 Degree of Multiprogramming

The **degree of multiprogramming** is the number of processes/jobs held in main memory at the same time. A higher degree generally means more overlap between CPU and I/O activity and therefore better CPU utilization — but pushing it too high can exhaust memory and hurt performance.

3.4 Advantages

- High CPU utilization — the CPU is rarely idle.
- Better throughput — more jobs complete per unit time.
- Efficient, economical use of overall system resources.

3.5 Disadvantages

- Requires a CPU-scheduling algorithm, adding design complexity and overhead.
- No guarantee of fast response — without careful scheduling a job may wait a long time; not well-suited to interactive use on its own.
- Higher memory requirement, since multiple jobs are resident simultaneously.

Example: early batch-processing systems such as IBM OS/360 introduced multiprogramming.

4.1 Concept

Multitasking — an extension of multiprogramming in which the OS rapidly switches the CPU between multiple tasks (processes), giving the illusion that they are all running simultaneously. Each task is given a very small slice of CPU time, called a **time slice / quantum**, before the CPU is switched to the next task.

The key goal of multitasking is fast, frequent switching so that **interactive users get quick response times** — unlike plain batch multiprogramming, which optimizes throughput rather than responsiveness.

4.2 Types of Multitasking

(a) Preemptive Multitasking — the OS itself decides when to take the CPU away from a running task (typically using a timer interrupt) and hand it to another task — even if the running task has not finished or voluntarily given up the CPU. This guarantees fairness and is used in all modern OSes (Windows, Linux, macOS).

(b) Cooperative (Non-preemptive) Multitasking — a running task must voluntarily yield control of the CPU. The OS cannot forcibly interrupt it. Risk: a single misbehaving or hung task can freeze the entire system. Used in early systems (e.g. Windows 3.x, classic Mac OS).

4.3 Multiprogramming vs Multitasking (classic exam comparison)

Basis	Multiprogramming	Multitasking
Primary goal	Maximize CPU utilization.	Provide fast response / time-sharing among tasks or users.
When CPU switches	Mainly when the running job needs to wait for I/O (less frequent).	On expiry of a small, fixed time quantum (frequent, regular switching).
Focus	Batch throughput.	User interactivity and responsiveness.
User interaction	Usually no direct interaction (batch jobs).	Direct, interactive users are the norm.
Typical systems	Early batch operating systems.	Modern time-sharing / desktop operating systems.

4.4 Relation to Time-Sharing Systems

Multitasking is the underlying mechanism that makes **time-sharing systems** possible — systems in which multiple users share the same computer interactively, each apparently getting their own share of the CPU at once, purely because the CPU switches between them fast enough that the delay is imperceptible to a human user.

5.1 Why CPU Scheduling Is Needed

In any multiprogrammed system, several processes sit in the **ready queue** at the same time, all competing for the one (or few) CPU(s) available. The **CPU scheduler** is the part of the OS that decides which ready process gets the CPU next, with the aim of maximizing CPU utilization and throughput while keeping waiting and response times low.

5.2 The CPU–I/O Burst Cycle

Process execution consists of a cycle of **CPU bursts** (periods of computation) alternating with **I/O bursts** (periods waiting for input/output to complete): CPU burst → I/O burst → CPU burst → ... → termination. Scheduling algorithms are built around predicting/handling these CPU bursts efficiently.

5.3 Preemptive vs Non-Preemptive Scheduling

Non-preemptive — once the CPU is allocated to a process, that process keeps the CPU until it either terminates or voluntarily switches to the waiting state (e.g. to request I/O). Example: FCFS.

Preemptive — the OS can forcibly take the CPU away from a running process — e.g. when a higher-priority process arrives, or when the running process's time quantum expires. Examples: Round Robin, preemptive Priority scheduling, and SVR3-style UNIX scheduling.

5.4 Types of Schedulers

Scheduler	Role	Frequency
Long-term (Job Scheduler)	Selects which jobs are admitted from the job pool into main memory / the ready queue; controls the degree of multiprogramming.	Invoked infrequently (minutes)
Short-term (CPU Scheduler)	Selects which of the ready, in-memory processes is given the CPU next.	Invoked very frequently (milliseconds) — must be fast
Medium-term	Temporarily removes ("swaps out") a process from memory to reduce the degree of multiprogramming, and later swaps it back in.	Invoked occasionally, as memory pressure demands

5.5 Scheduling Criteria / Performance Metrics

Criterion	Meaning	Goal
CPU Utilization	Percentage of time the CPU is actually busy.	Maximize
Throughput	Number of processes completed per unit time.	Maximize

Criterion	Meaning	Goal
Turnaround Time (TAT)	Completion Time – Arrival Time.	Minimize
Waiting Time (WT)	Turnaround Time – Burst Time (time spent only waiting in the ready queue).	Minimize
Response Time	Time of first CPU allocation – Arrival Time.	Minimize

5.6 The Dispatcher

The **dispatcher** is the module that actually hands the CPU over to the process chosen by the short-term scheduler. This involves a context switch, a switch to user mode, and jumping to the correct location in the user program. The time this takes is called **dispatch latency**.

6.1 Concept

FCFS — the simplest CPU-scheduling algorithm. Processes are executed strictly in the order in which they arrive in the ready queue — exactly like people being served in the order they join a queue at a ticket counter. It is implemented with a simple FIFO (First-In-First-Out) queue.

6.2 Characteristics

- Non-preemptive — once a process starts, it runs to completion.
- Very easy to understand and implement.
- Average waiting time is often poor, especially if a long process happens to arrive first.

6.3 Algorithm

1. Arrange processes in order of arrival time.
2. Allocate the CPU to the process at the head of the queue.
3. Let it run to completion (no preemption).
4. Move to the next process in the queue and repeat.

6.4 Solved Example

Consider four processes with the following arrival times (AT) and CPU burst times (BT):

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Since FCFS runs processes strictly in arrival order, the schedule is P1 → P2 → P3 → P4:

P1 0–5	P2 5–8	P3 8–16	P4 16–22
------------------	------------------	-------------------	--------------------

Gantt chart — each block shows the process and its [start–end] time.

Working out Completion Time (CT), Turnaround Time (TAT = CT – AT) and Waiting Time (WT = TAT – BT):

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	8	7	4

Process	AT	BT	CT	TAT	WT
P3	2	8	16	14	6
P4	3	6	22	19	13
Average				11.25	5.75

Avg Turnaround Time = $(5+7+14+19) / 4 = 11.25$

Avg Waiting Time = $(0+4+6+13) / 4 = 5.75$

6.5 Advantages

- Simple to understand and implement.
- Fair in the sense of arrival order — every process eventually runs; no starvation.
- No overhead of complex scheduling decisions.

6.6 Disadvantages

- **Convoy Effect:** if a long CPU-bound process is at the front of the queue, every short process behind it must wait a long time — this drags down the average waiting time badly.
- Not suitable for time-sharing/interactive systems, since it is non-preemptive and gives poor response time.
- Average waiting time is generally higher than in other scheduling algorithms.

7.1 Concept

Round Robin — an algorithm designed especially for time-sharing systems. Every process is given a small, fixed unit of CPU time called a **time quantum** (or time slice), typically 10–100 ms. The ready queue is treated as a circular queue.

7.2 Algorithm

1. Arrange processes in the ready queue in order of arrival (FIFO).
2. The scheduler takes the process at the head of the queue and runs it for **one time quantum**.
3. If the process finishes within that quantum, it leaves the system.
4. If it does **not** finish, it is preempted, sent to the **back** of the ready queue, and the CPU is given to the next process in line.
5. Repeat until every process has completed.

7.3 Why the Time Quantum Matters

- Too **large** a quantum → Round Robin starts to behave like FCFS → poor response time.
- Too **small** a quantum → excessive context switching → the CPU wastes more time switching between processes than actually executing them.
- Rule of thumb often quoted: aim for roughly 80% of CPU bursts to be shorter than the time quantum.

7.4 Solved Example (Time Quantum = 4)

Using the same four processes as before, with a time quantum of 4:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Step-by-step trace (queue shown after each preemption/completion):

- t=0–4: run P1 (rem. 5–4=1). During this, P2, P3, P4 arrive → queue becomes [P2, P3, P4, P1(1)]
- t=4–7: run P2 (burst 3, finishes exactly) → queue: [P3, P4, P1(1)]
- t=7–11: run P3 (rem. 8–4=4) → queue: [P4, P1(1), P3(4)]
- t=11–15: run P4 (rem. 6–4=2) → queue: [P1(1), P3(4), P4(2)]
- t=15–16: run P1 (rem. 1, finishes) → queue: [P3(4), P4(2)]
- t=16–20: run P3 (rem. 4, finishes) → queue: [P4(2)]
- t=20–22: run P4 (rem. 2, finishes) → done

P1 0-4	P2 4-7	P3 7-11	P4 11-15	P1 15-16	P3 16-20	P4 20-22
-----------	-----------	------------	-------------	-------------	-------------	-------------

Gantt chart for Round Robin, time quantum = 4.

Process	AT	BT	CT	TAT	WT
P1	0	5	16	16	11
P2	1	3	7	6	3
P3	2	8	20	18	10
P4	3	6	22	19	13
Average				14.75	9.25

Avg Turnaround Time = $(16+6+18+19) / 4 = 14.75$

Avg Waiting Time = $(11+3+10+13) / 4 = 9.25$

Compare with FCFS above: RR gives much better response time (P2 finishes by t=7) at the cost of higher average turnaround/waiting time here, since every process is interrupted and repeatedly re-queued.

7.5 Advantages

- Fair — every process gets an equal, regular opportunity at the CPU.
- Good response time, which makes it ideal for interactive/time-sharing systems.
- No starvation.

7.6 Disadvantages

- Average waiting time is often higher than algorithms like SJF.
- Performance depends heavily on choosing a good time quantum size.
- Overhead from frequent context switching if the quantum is set too small.

8.1 Background

SVR3 refers to UNIX System V Release 3. Both SVR3 and 4.3BSD UNIX used a very similar **traditional UNIX scheduling algorithm**, designed for a time-sharing, interactive environment. The design goal was to give good response time to interactive users while making sure low-priority background jobs are never completely starved of the CPU.

8.2 Key Characteristics

- Priority-based and **preemptive**.
- Uses **multilevel feedback queues**, with Round Robin scheduling *within* each priority level.
- Priorities are dynamically recalculated **once every second** for every process.
- Uses **one-second preemption**: even if no process finishes, the scheduler reassesses priorities and queue positions every second.

8.3 The Counter-Intuitive Priority Numbering

Important exam point — in traditional UNIX scheduling, a **higher numeric priority value means LOWER actual priority**. The process with the *smallest* number runs first. This is the opposite of how people usually think of “priority 1, 2, 3...” and is a favourite point examiners ask about directly.

8.4 Per-Process Scheduling Data

Field	Meaning
p_cpu	A running measure of the process's recent CPU usage / execution history.
p_pri (usr_pri)	The process's current scheduling priority (recalculated every second).
nice	A user/administrator-controllable value used to bias a process's priority up or down. A process can voluntarily “be nice” and lower its own scheduling priority.

8.5 The Scheduling Formula

On every clock tick, the currently running process's **p_cpu** is incremented by 1. Once every second, for every process, the scheduler performs two steps:

Step 1 (decay usage): $p_cpu = p_cpu / 2$

Step 2 (recompute priority): $p_priority = (p_cpu / 4) + PUSER + (2 \times nice)$

PUSER = base priority for user processes. Halving p_cpu each second lets the formula “forget” old CPU usage, so a process is not punished forever for CPU it consumed in the distant past.

Effect of the formula: a process that has used a lot of CPU recently builds up a large p_cpu → larger numeric priority value → **lower** scheduling priority (it runs later). A process that has mostly been waiting on I/O has its p_cpu decay toward zero → smaller numeric value → **higher** scheduling priority (it runs sooner). This naturally favours **I/O-bound / interactive** processes over CPU-bound ones — exactly the behaviour the algorithm was designed to produce.

8.6 Priority Bands

Beyond ordinary user processes, the kernel itself reserves several fixed priority bands, listed here in **decreasing** order of priority (the first band always runs before a later one):

Band	Name	Purpose
1 (highest)	Swapper	Kernel-level memory swapping activity.
2	Block I/O device control	Managing block-oriented devices such as disks.
3	File manipulation	Kernel-level file system operations.
4	Character I/O device control	Managing character-oriented devices (e.g. terminals).
5 (lowest)	User processes	Ordinary user programs — priority computed by the formula above.

These bands exist so the kernel can respond quickly to system calls and manage device access efficiently before any ordinary user code is allowed to run.

8.7 Multilevel Feedback + Round Robin

Because priorities are recomputed continually, a process can move between priority queues over time — this is the “feedback” part of *multilevel feedback queues*. Within any single priority level, processes are scheduled Round Robin.

8.8 Worked Numeric Example

Take $PUSER = 60$ and $nice = 0$ for two processes: **A**, which is CPU-bound and runs almost continuously (gaining ~20 usage-units per second while running), and **B**, which is I/O-bound and only briefly touches the CPU (gaining ~2 usage-units per second). Applying $p_cpu = (old\ p_cpu + new\ usage) / 2$ each second:

Sec.	A: p_cpu	A: priority	B: p_cpu	B: priority
0 (start)	0	60	0	60
1	10 $(=(0+20)/2)$	62	1 $(=(0+2)/2)$	60
2	15 $(=(10+20)/2)$	63	1.5 $(=(1+2)/2)$	60
3	17.5 $(=(15+20)/2)$	64	1.75 $(=(1.5+2)/2)$	60

Observation — process A's priority *number* keeps climbing (60→62→63→64), meaning A becomes steadily **less** favoured the more CPU it consumes. Process B's priority number barely moves, so B stays highly favoured. This is exactly how the algorithm pushes CPU-hungry processes down and keeps interactive/I/O-bound processes responsive.

(Figures here use simplified, illustrative usage-per-second numbers to demonstrate the mechanism — the concept and formula are what examiners look for, not a specific clock-tick rate.)

8.9 Advantages

- Relatively simple and effective — only integer arithmetic is needed.
- Works well for general-purpose, single-processor interactive systems.
- Naturally favours I/O-bound (interactive) processes over CPU-bound ones, giving good response time.
- Prevents starvation — the periodic decay ensures no process is permanently stuck with a bad priority.

8.10 Disadvantages

- Significant overhead in large systems, since priorities are recomputed for *every* process, every second — this does not scale well.
- Response time is not strictly guaranteed (it is not a real-time algorithm).
- The formula's constants (PUSER, the decay factor, the nice weight) are empirically tuned rather than mathematically optimal.

SECTION 9

Quick Comparison — FCFS vs Round Robin vs SVR3

Basis	FCFS	Round Robin	SVR3
Preemptive?	No	Yes (on quantum expiry)	Yes (on priority change / 1-sec tick)
Basis of selection	Arrival order	Arrival order + fixed time quantum	Dynamic priority (recent CPU usage)
Best suited for	Batch systems	Time-sharing / interactive systems	General-purpose UNIX interactive systems
Response time	Poor	Good	Good (favours I/O-bound processes)
Starvation?	No (strict FIFO order)	No	No (usage decay prevents it)
Overhead	Very low	Moderate–high (context switches)	Higher (recompute every process, every second)
Key weakness	Convoy effect	Performance sensitive to quantum size	Doesn't scale to very large process counts

Exam tip — if a question asks you to “compare” these algorithms, always anchor your answer to the five scheduling criteria from Section 5.5 (CPU utilization, throughput, turnaround time, waiting time, response time) — examiners are typically checking whether you can connect the algorithm's mechanism to its effect on these metrics.

Formula Sheet

Turnaround Time (TAT) = Completion Time – Arrival Time
Waiting Time (WT) = Turnaround Time – Burst Time
Response Time = Time of first CPU allocation – Arrival Time
SVR3 decay: $p_cpu = p_cpu / 2$
SVR3 priority: $p_priority = (p_cpu / 4) + PUSER + (2 \times nice)$

Short-Answer Revision (2-mark style)

Q. Define Operating System.

System software that acts as an intermediary between the user/application programs and computer hardware, managing all resources and providing common services to programs.

Q. List the three main objectives of an OS.

Convenience, Efficiency, and Ability to Evolve.

Q. Differentiate Kernel and Shell in one line each.

Kernel: the core of the OS that directly manages hardware resources. Shell: the outer command interpreter that lets the user interact with the kernel.

Q. What is the degree of multiprogramming?

The number of processes/jobs held in main memory at the same time.

Q. Define Turnaround Time and Waiting Time.

Turnaround Time = Completion Time – Arrival Time. Waiting Time = Turnaround Time – Burst Time.

Q. What is the Convoy Effect?

In FCFS, a long CPU-bound process at the head of the queue forces every shorter process behind it to wait an unfairly long time, hurting average waiting time.

Q. Why is the time quantum important in Round Robin?

Too large a quantum makes RR behave like FCFS (poor response time); too small a quantum causes excessive context-switching overhead. It must be tuned for a balance.

Q. What does “higher priority number = lower priority” mean in SVR3/UNIX scheduling?

Traditional UNIX scheduling numbers priorities the opposite of everyday intuition — the process with the smallest numeric value is scheduled first, not the largest.

Q. Name the priority bands used in traditional UNIX (SVR3) scheduling.

In decreasing order of priority: Swapper, Block I/O device control, File manipulation, Character I/O device control, User processes.

Q. What is the role of the ‘nice’ value in SVR3 scheduling?

A user/administrator-controllable value that biases a process's computed priority up or down, letting a process voluntarily lower its own scheduling priority.

Long-Answer Revision (10-mark style — answer structure)

For long-answer questions, use these as checklists of everything the answer should cover:

Q. Explain the objectives and functions of an operating system.

Structure: (1) Define OS, mention the two views (user view / system view). (2) Explain all three objectives — Convenience, Efficiency, Ability to Evolve — one paragraph each. (3) List and briefly explain all ten functions from Section 1.3, ideally as a table. (4) Optionally include the layered diagram (Users → Programs → OS → Hardware).

Q. What is CPU scheduling? Explain FCFS and Round Robin with suitable examples.

Structure: (1) Define CPU scheduling and why it's needed. (2) List the scheduling criteria. (3) Explain FCFS — definition, algorithm, one solved numerical with Gantt chart, advantages/disadvantages (mention convoy effect). (4) Explain Round Robin — definition, role of time quantum, algorithm, one solved numerical with Gantt chart, advantages/disadvantages.

Q. Explain the traditional UNIX (SVR3) scheduling algorithm with its formula.

Structure: (1) Background — SVR3 & 4.3BSD, goal of good interactive response without starvation. (2) The counter-intuitive priority numbering. (3) The formula (decay + priority recompute) with a labelled diagram/worked trace. (4) The five priority bands, in order. (5) Advantages and disadvantages.

Q. Differentiate between Multiprogramming and Multitasking operating systems.

Structure: define both, then present the five-row comparison table from Section 4.3 (goal, switching trigger, focus, user interaction, typical systems), with one example system for each.

Glossary of Key Terms (alphabetical)

Term	Meaning
Burst Time	The amount of CPU time a process requires to complete its execution (or a segment of it).
Context Switch	Saving the state of a running process and loading the state of another, so the CPU can switch between them.
Convoy Effect	In FCFS, short processes queued behind one long process suffer long waiting times.
CPU Utilization	The percentage of time the CPU is actively executing (not idle).
Degree of Multiprogramming	The number of processes held in main memory at the same time.
Dispatcher	The module that hands the CPU to the process chosen by the short-term scheduler.
Gantt Chart	A timeline diagram showing which process runs on the CPU during which time interval.
Kernel	The core of the OS with privileged, direct control of hardware.
Preemptive Scheduling	Scheduling in which a running process can be forcibly interrupted before it finishes.
Ready Queue	The set of processes in memory that are ready to run and waiting for the CPU.
Response Time	Time from a process's submission to its first CPU allocation / first output.
Shell	The command-interpreter interface between the user and the kernel.
System Call	The programming interface through which a user-mode program requests a kernel-mode service.
Throughput	The number of processes completed per unit time.
Time Quantum	The small, fixed slice of CPU time given to a process in Round Robin scheduling.
Turnaround Time	Total time from a process's submission to its completion.
Waiting Time	Total time a process spends waiting in the ready queue (excluding execution/I/O).

Study Tips Before the Exam

- Re-derive the FCFS and Round Robin numericals yourself with a **different** time quantum (e.g. quantum = 2 or 3) — this is the single best way to make sure you actually understand the mechanism rather than having memorised one example.
- For SVR3, focus on being able to *state* the formula and explain *why* higher p_cpu leads to a less-favourable (higher) priority number — this conceptual explanation is usually worth more marks than the arithmetic itself.

- The Multiprogramming-vs-Multitasking and FCFS-vs-RR-vs-SVR3 comparison tables in this guide are very likely exam material — be able to reproduce them from memory.
- Revisit the Formula Sheet at the start of Section 10 the night before the exam as your final pass.
- Cross-check these notes against your class notes/textbook — different instructors sometimes phrase a definition slightly differently, and it's worth matching your instructor's wording where you can.

End of Unit 1 Study Notes. Good luck with your mid-semester exam!